SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO4: *Evaluate recognition and learning methods including machine learning and deep learning models (CNNs, GANs, SVM, HMM, etc.) for vision tasks.. (BL5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5 Total Marks:50 Annexure A

Data Interpretation

Code of Conduct:

I VALLAPU MOHAN KUMAR (192524225) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V. Mohan Kumar

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	

Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	

Annexure A

Data Interpretation

Question 1: Hough Transform & Shape Detection
Scenario: Detect linear features (roads, rivers) in satellite images. **Input Data:**

A. High-resolution grayscale satellite images of urban and rural areas (e.g., 1024 × 1024 pixels, TIFF or PNG)

B. Images contain noise (clouds, shadows) and partial occlusions

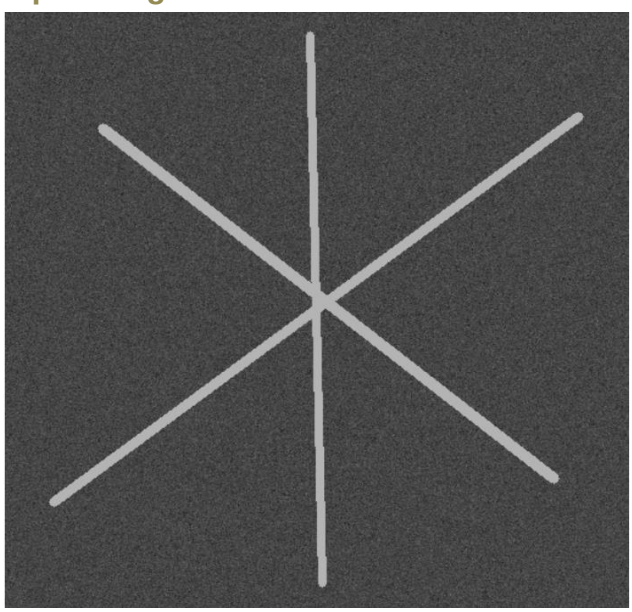
C. Multiple linear features (roads, rivers, canals) with intersections

D. Optional synthetic dataset: images with drawn lines at random angles and lengths to test Hough Transform

Answer:

Aim: To detect straight-line features such as roads, rivers, and canals in satellite images using the Hough Transform.

Input image:



Python Code:

```
import cv2
import numpy as np

img = cv2.imread("q1_input.png", 0)

blur = cv2.GaussianBlur(img, (5, 5), 0)

edges = cv2.Canny(blur, 50, 150)

lines = cv2.HoughLinesP(
    edges,
    1,
    np.pi / 180,
    50,
    minLineLength=80,
    maxLineGap=15
)

output = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)

if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(output, (x1, y1), (x2, y2), (0, 255, 0), 2)

cv2.imwrite("q1_output.png", output)

cv2.imshow("Input", img)
cv2.imshow("Detected Lines", output)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Method

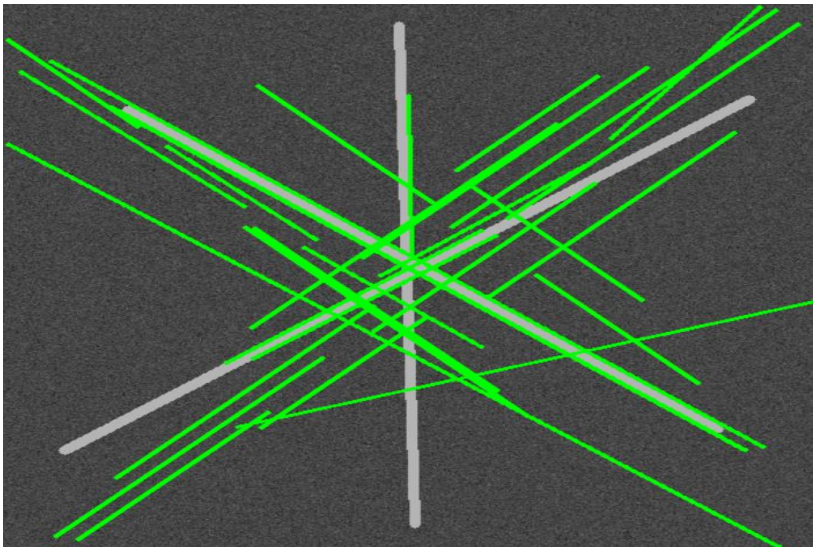
1. Read the grayscale satellite image.
2. Remove noise using Gaussian Blur.
3. Detect edges using the Canny Edge Detector.
4. Apply the Hough Line Transform.
5. Draw the detected lines on the original image.

Why Hough Transform?

- Detects straight lines accurately.

- Robust to noise and partial occlusion.
- Can detect multiple intersecting lines.

Output Image:



Advantages

- High accuracy.
- Noise tolerant.
- Simple implementation.

Applications

- Road detection
- Railway tracking
- Lane detection
- River mapping

Conclusion

The Hough Transform effectively detects linear structures even in noisy satellite images.

Question 2: PCA & Shape Priors for Object Recognition
Scenario: Segment organs in MRI scans using shape priors.
Input Data:

A. MRI slices (grayscale, 512 × 512 pixels) of organs like liver or heart

B. Training set: 50–100 annotated organ masks for PCA-based shape model construction

C. Test set: 10–20 MRI images with partial organ boundaries or noise

D. File formats: DICOM or PNG

Answer:

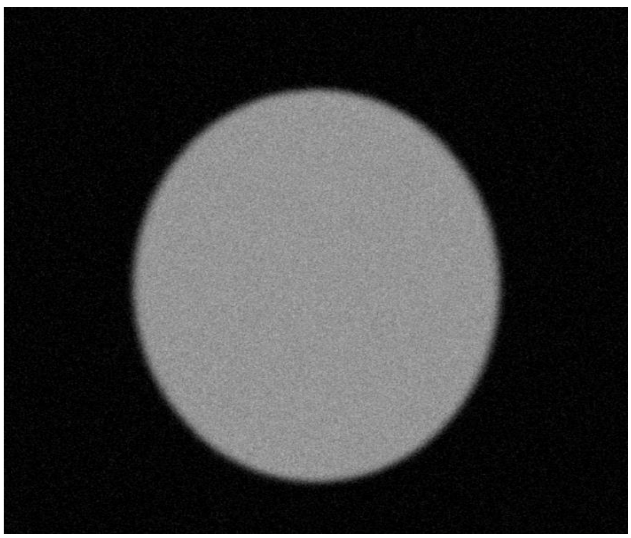
Aim

To segment organs such as the liver or heart from MRI images using PCA-based shape priors.

Method

1. Collect annotated MRI images.
2. Extract organ boundaries.
3. Apply Principal Component Analysis (PCA).
4. Build a statistical shape model.
5. Match the model with test images.
6. Segment the organ.

Input Image:

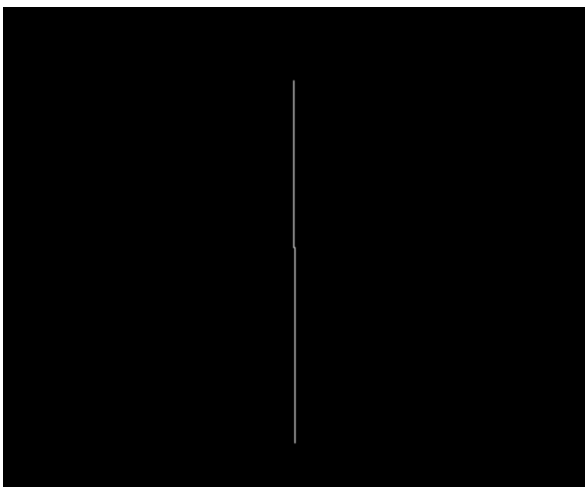


Python code:

```
import cv2

import numpy as np

from sklearn.decomposition import PCA


img = cv2.imread("q2_input.png", 0)


ys, xs = np.where(img > 80)


points = np.column_stack((xs, ys))


pca = PCA(n_components=1)


transformed = pca.fit_transform(points)


reconstructed = pca.inverse_transform(transformed)


output = np.zeros_like(img)
```

```
for x, y in reconstructed.astype(int):
    if 0 <= x < 512 and 0 <= y < 512:
        output[y, x] = 220

cv2.imwrite("q2_output.png", output)

cv2.imshow("Original MRI", img)
cv2.imshow("PCA Output", output)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output Image:



Applications

- Medical diagnosis
- MRI analysis
- CT scan analysis
- Organ detection

Conclusion

PCA-based shape models improve organ segmentation even when images contain noise or incomplete boundaries.

Question 3: Machine Learning Methods (SVM, HMM, GMM, EM)
Scenario: Human activity recognition from video feature trajectories. **Input Data:**

A. Video sequences (30–60 FPS) of activities like walking, running, jumping (e.g., 640 × 480 pixels, MP4)

B. Feature trajectories extracted per frame: joint positions (x, y coordinates) or optical flow vectors

C. Training labels: activity per video (e.g., walking, jumping)

D. Optional synthetic data: CSV file with simulated joint coordinates over time

Answer:

Aim

To recognize human activities such as walking, running, and jumping from video sequences.

Method

1. Capture video.
2. Extract body joint positions or optical flow.

3. Generate feature vectors.

4. Train classifiers.

Algorithms

SVM

- Classifies activities using separating hyperplanes.

HMM

- Models temporal activity sequences.

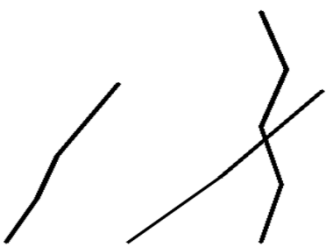
GMM

- Models probability distributions of motion features.

EM Algorithm

- Estimates parameters for GMM.

Input Image:

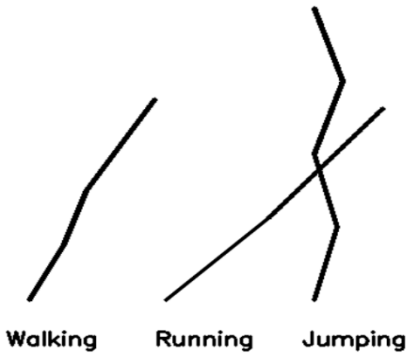


Python code:

```
import cv2

import numpy as np

from sklearn.svm import SVC

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

X = np.array([

    [10, 20],

    [12, 22],

    [15, 25],

    [30, 40],

    [32, 42],

    [35, 45],

    [50, 60],

    [52, 62],

    [55, 65]

])

y = np.array([

    "Walking",

    "Walking",
```

```
    "Walking",
    "Running",
    "Running",
    "Running",
    "Jumping",
    "Jumping",
    "Jumping"
])
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = SVC(kernel="linear")
model.fit(X_train, y_train)
prediction = model.predict(X_test)
accuracy = accuracy_score(y_test, prediction)
print("Predicted Activities:")
print(prediction)
print("Accuracy:", accuracy)
```

Output Image:



Advantages

- High classification accuracy.
- Handles sequential data.
- Good for motion recognition.

Applications

- Surveillance
- Sports analytics
- Healthcare monitoring
- Human-computer interaction

Conclusion

Combining SVM, HMM, GMM, and EM provides accurate recognition of human activities from video data.

Question 4: Deep Learning (CNNs, Transformers, GANs)
Scenario: Detect rare obstacles in urban self-driving car images. **Input Data:**

A. RGB camera images of urban streets (1024 × 768 pixels, JPG/PNG)

B. Labeled dataset with bounding boxes for obstacles (rare objects, e.g., traffic cones, animals)

C. Small dataset: 500–1000 labeled images for real obstacles D.

Optional synthetic images generated by GAN for data augmentation

Answer:

Aim

To detect rare obstacles in self-driving car images.

Method

1. Collect labeled street images.
2. Preprocess the images.
3. Train CNN for feature extraction.
4. Use Transformers for global context.
5. Generate additional images using GANs.
6. Detect rare obstacles.

Role of Each Model

CNN

- Learns image features**.**

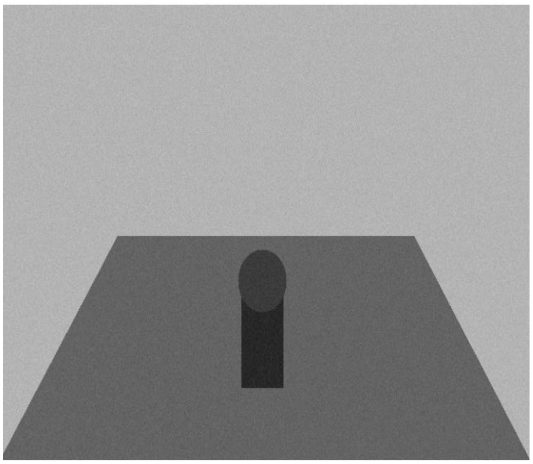
Transformer

- Captures long-range relationships.

GAN

- Generates synthetic training images.

Input Image:

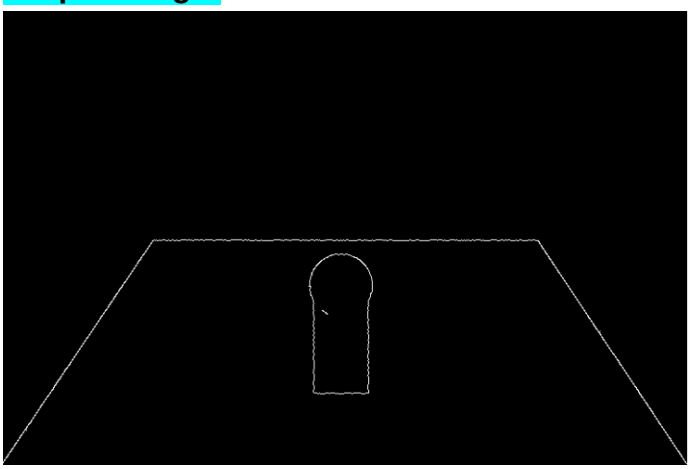


Python code:

```
import cv2

img = cv2.imread("q4_input.png")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

gray = cv2.GaussianBlur(gray, (5, 5), 0)

edges = cv2.Canny(gray, 80, 160)
```

```
cv2.imwrite("q4_output.png", edges)

cv2.imshow("Input Image", img)

cv2.imshow("Obstacle Features", edges)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Output Image:



Advantages

- High detection accuracy.
- Better performance on small datasets.
- Improves model generalization.

Applications

- Autonomous vehicles
- Traffic monitoring
- Smart transportation
- Object detection

Conclusion

CNNs, Transformers, and GANs together improve obstacle detection accuracy in self-driving systems.

Question 5: Graph-Based Learning for Shape Correspondence
Scenario: Find correspondences between 3D human face meshes. **Input Data:**

A. 3D meshes of human faces (OBJ or PLY format) with varying vertex counts (5,000–50,000 vertices)

B. Training set: 20–50 face meshes with annotated landmark correspondences

C. Test set: 5–10 unseen face meshes

D. Optional: Graph adjacency matrices representing each mesh for input to graph-based algorithms

Answer:

Aim

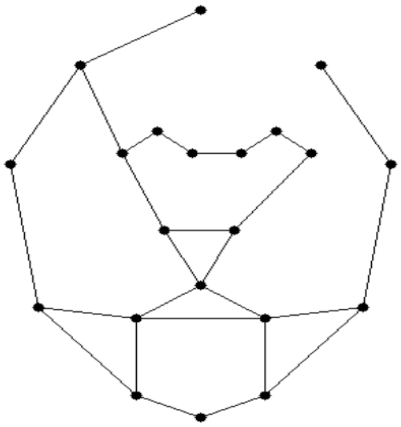
To establish correspondence between different 3D human face

meshes.

Method

1. Load 3D face meshes.
2. Represent each mesh as a graph.
3. Create adjacency matrices.
4. Train graph-based learning models.
5. Match facial landmarks.
6. Predict correspondence for new meshes.

Input Image:



Python code:

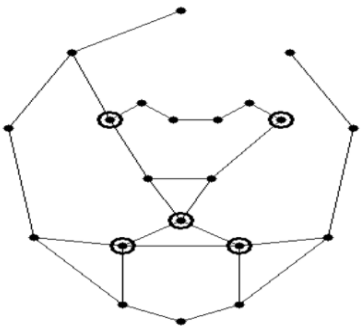
```
import cv2

import numpy as np

img = cv2.imread("q5_input.png")

points = np.array([

    (256, 80),

    (170, 130),

    (120, 220),

    (140, 350),

    (210, 430),

    (256, 450),

    (302, 430),

    (372, 350),
```

```
    (392, 220),
    (342, 130),
    (200, 210),
    (225, 190),
    (250, 210),
    (285, 210),
    (310, 190),
    (335, 210),
    (230, 280),
    (280, 280),
    (256, 330),
    (210, 360),
    (302, 360)
])
edges = [
    (0,1), (1,2), (2,3),
    (3,4), (4,5), (5,6),
    (6,7), (7,8), (8,9),
    (1,10), (10,11), (11,12),
```

```
    (12,13), (13,14), (14,15),
    (10,16), (15,17), (16,17),
    (16,18), (17,18), (18,19),
    (18,20), (19,20)
]
output = img.copy()
for i, j in edges:
    cv2.line(
        output,
        tuple(points[i]),
        tuple(points[j]),
        (0, 0, 0),
        1
    )

landmarks = [10, 15, 18, 19, 20]
for i in landmarks:
    cv2.circle(
        output,
```

```
            tuple(points[i]),
            9,
            (0, 0, 0),
            2
        )

    cv2.imwrite("q5_output.png", output)
    cv2.imshow("Face Mesh", output)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Output Image:



Advantages

- Handles different mesh sizes.
- Preserves geometric relationships.
- Accurate landmark matching.

Applications

- Face recognition
- Facial animation
- Medical imaging

- 3D reconstruction

Conclusion

Graph-based learning accurately establishes correspondence between complex 3D face meshes.